

Summary

This polish pass made WealthWise more production-ready without changing the core product

Idea, The app is still a Streamlit finance tracker for Indian students and retail

investors, but the code is now easier to read, safer to run, more truthful in its

documentation, and better structured around the actual data flow.

What Changed

- Reworked `main.py` into smaller functions for upload handling, dashboard rendering,

labeling, raw data display, and AI advisor interactions.

- Added safer session-state handling so Streamlit reruns do not constantly reprocess

the same uploaded file or rebuild embeddings unnecessarily.

- Improved file ingestion by supporting both CSV and XLSX paths correctly and showing

clearer errors when required columns are missing.

- Fixed the default rules filename from `seed_rules.json` to the actual

`seeds_rules.json`.

- Separated seed rules from user rules so saving a label no longer writes the built-

in rules back into `user_rules.json`.

- Standardized important categories like `'UNCATEGORIZED'`, `'INTERNAL TRANSFER'`, and

`'FRIENDS PAYMENTS'` through shared config constants.

- Made embedding and GrogLangChain setup lazy, so expensive models and external

clients are only initialized when needed.

- Cleaned the ChromaDB transaction store before writing the current session's

embeddings, reducing stale retrieval results.

- Updated the README so it reflects the real code, real setup, real privacy behavior,

and current project structure.

- Added `'openpyxl'` to requirements because the app accepts `'xlsx'` uploads.

- Replaced the old RAG test script with a simple dependency smoke check.

Why These Changes Help

- Reliability: Missing upload columns now produce clear errors instead of confusing

downstream crashes.

- Maintainability: The Streamlit app is split into focused functions, making future

changes much easier.

- Performance: Embeddings are cached by transaction fingerprint, so unchanged data is

not repeatedly embedded.

- Correctness: Rule loading now points to the actual seed rules file and keeps

seed/user rule responsibilities separate.

• Better RAG quality: Clearing old transaction embeddings helps the AI advisor answer

from the current dataset instead of stale data.

• Better onboarding: The README now explains setup, usage, validation, project

structure, and privacy accurately

• Cleaner failure modes: The AI advisor now reports a missing Groq key clearly

instead of failing during import.

High-Level Code Flow

1. 'main.py' starts the Streamlit app, initializes session state, and renders the

sidebar inputs.

2. The user uploads a CSV/XLSX file, or the app falls back to generated mock

transactions.

3. 'processor.py' maps messy export headers into 'Date', 'Description', and 'Amount'.

4. 'processor.py' cleans amounts, parses dates, extracts merchant names, flags

internal transfers, investments, personal payments, and reimbursements.

5. 'classifier.py' categorizes merchants using exact rules, fuzzy matching, and then

ML fallback when enough labeled examples exist.

6. 'embedding.py' turns each transaction into a short text sentence and creates

vector embeddings.

7. 'vector_store.py' stores those embeddings in ChromaDB for semantic search.

8. 'main.py' renders the dashboard, labeling workflow, raw transaction table, and AI

advisor tab.

9. If the user asks the AI advisor a question, 'rag_advisor.py' retrieves relevant

transactions and optional PDF knowledge, then sends that context to Groq through

LangChain.

Theory Behind The App

WealthWise combines deterministic finance rules with lightweight machine learning and

retrieval-augmented generation.

The deterministic layer handles things that should be predictable: header mapping, amount

cleaning, transfer detection, investment detection, and reimbursement matching. This gives

the app a stable foundation.

The classification layer uses a hybrid approach. Exact rules are preferred because they

are reliable. Fuzzy matching handles merchant spelling and formatting variations. The

Random Forest classifier acts as a final fallback for unseen merchants, but only when

there is enough training data and confidence.

answer from generic memory, the app first retrieves transaction and document snippets from

ChromaDB. The LLM then answers using that retrieved context, which makes responses more

grounded in the user's actual spending.

Files Most Affected

- `main.py`: Streamlit structure, session-state flow, upload handling, dashboard.
- `labeling.py`: AI advisor.
- `processor.py`: Safer ingestion, standard categories, reimbursement configuration.
- `classifier.py`: Cleaner rule loading/saving, safer ML fallback.
- `embedding.py`: Lazy embedding model loading.
- `vector_store.py`: Current-session transaction storage and retrieval.
- `knowledge_base.py`: Lazy embedding usage and safer PDF text extraction.
- `rag_advisor.py`: Lazy Groq chain creation and cleaner API-key error.
- `README.md`: Updated to match the real application.
- `requirements.txt`: Added XLSX support dependency.
- `test_rag_advisor.py`: Converted into a practical smoke check.

Suggested Commit Message

Polish WealthWise app structure and update documentation

Suggested Commit Body

- Refactor Streamlit app into focused render and data-preparation functions
- Harden transaction ingestion, classification, and rule persistence
- Lazy-load embeddings and Groq RAG chain for cleaner runtime behavior
- Refresh README to match actual app flow, setup, privacy, and validation
- Add XLSX dependency and replace RAG script with dependency smoke check